


```
from growapi import GrowAPI

# GrowAPI Credentials (Replace with your actual credentials,
API_AUTH_TOKEN = "your_token"

# Initialize GrowAPI
grow = GrowAPI(API_AUTH_TOKEN)

oco_response = grow.create_smart_order(
    smart_order_type=grow.SMART_ORDER_TYPE_OCO,
    reference_id="oco-ref-unique565",
    segment=grow.SEGMENT_FNO,
    trading_symbol="NIFTY25CO24000ICE",
    quantity=50,
    product_type=grow.PRODUCT_MIS,
    exchange=grow.EXCHANGE_NSE,
    duration=grow.VALIDITY_DAY,
    # OCO-specific parameters
    net_position_quantity=50,
    transaction_type=grow.TRANSACTION_TYPE_SELL,
    targets=[
        {
            "trigger_price": "120.50",
            "order_type": grow.ORDER_TYPE_LIMIT,
            "price": "121.00"
        },
    ],
    stop_loss={
        "trigger_price": "95.00",
        "order_type": grow.ORDER_TYPE_STOP_LOSS_MARKET,
        "price": None
    }
)
```

Parameters

Name	Type	Description
smart_order_type *	str	Setto OCO to create a One-Cancels-Other smart order (target + stop-loss). Example: OCO .
reference_id *	str	User-provided alphanumeric string (8-20 characters) that serves as an idempotency key, with at most two hyphens (-) allowed.
segment *	str	Segment for which the order will be placed. See Segment . Examples: FNO , CASH .
trading_symbol *	str	Trading Symbol of the instrument as defined by the exchange
quantity *	int	Total quantity for both legs. Must be $\leq$ abs (net_position_quantity) . Example: 50 .
net_position_quantity *	int	Your current net position in this symbol. Used to derive leg directions and validate quantity. Example: 50 .
transaction_type *	str	Direction of protection/exit for your position. See Transaction type . Examples: BUY , SELL .
target *	dict	Take-profit leg details
target["trigger_price"] *	str	Take-profit trigger price (decimal string). Example: "120.50" .
target["order_type"] *	str	Order type for the target leg. See Order type . Examples: LIMIT , MARKET .
target["price"]	str	Target leg limit price (required if order_type = LIMIT) . Example: "121.00" .
stop_loss *	dict	Stop-loss leg details
stop_loss["trigger_price"] *	str	Stop-loss trigger price (decimal string). Example: "95.00" .
stop_loss["order_type"] *	str	Order type for the stop-loss leg. See Order type . Examples: SL , SL_M .
stop_loss["price"]	str	Stop-loss leg limit price (required if order_type = SL) . Example: "94.50" .
product_type *	str	Product for the OCO. See Product type . Note: For OCO in cash segment, only MTS is supported currently.
exchange *	str	Exchange for this instrument. See Exchange . Example: NSE .
duration *	str	Validity for both legs. See Validity . Example: DAY .

required parameters

"Important OCO orders are meant to exit an existing position."

- In the **F&O** segment you must already hold the contract.
- In the **CASH** segment, **OCO** is restricted to intraday (**MIS**) positions.

Response

```
{
  "smart_order_id": "0c0c1d2bc3",
  "smart_order_type": "OCO",
  "status": "ACTIVE",
  "exchange_symbol": "NIFTY25OCT24080CE",
  "exchange": "NIFTY",
  "quantity": 50,
  "product_type": "MIS",
  "duration": "DAY",
  "target": {
    "trigger_price": "120.50",
    "order_type": "LIMIT",
    "price": "121.00"
  },
  "stop_loss": {
    "trigger_price": "95.00",
    "order_type": "SL_M",
    "price": null
  },
  "is_cancellation_allowed": true,
  "is_modification_allowed": true,
  "created_at": "2025-09-30T07:00:00",
  "expire_at": null,
  "updated_at": null,
  "updated_at": "2025-09-30T07:00:00"
}
```

Schema reference: Smart Order Response (OCO).

Notes

- `quantity` must be $\leq \text{abs}(\text{net_position_quantity})$.
- If a leg executes, the other cancels automatically.

Modify Smart Order

Modify contracts differ by flow. Only the fields listed below are honoured; everything else is ignored or rejected. Use cancel + create when you need changes outside of these lists.

Modifiable Fields - GTT

- `quantity` - Updated order quantity
- `trigger_price` - Updated trigger price threshold
- `trigger_direction` - Updated direction

- * `order.order_type` - Updated order type
- * `order.price` - Updated limit price (required for `LIMIT` / `SL` types; set to `None` for `MARKET` / `SL_M`)
- * `child_legs` - Updated bracket order legs (optional; all child leg fields are modifiable if provided)

Modifiable Fields - OCO

- * `quantity` - Updated order quantity
- * `duration` - Updated validity
- * `product_type` - Updated product (e.g. `MIS` ↔ `NRML` for `FNO`; `CASH` OCO only supports `MIS`)
- * `target.trigger_price` - Updated profit target trigger price
- * `stop_loss.trigger_price` - Updated stop-loss trigger price

Modify GTT

```
from growwapl import GrowwAPI

# Groww API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

# Initialize Groww API
groww = GrowwAPI(API_AUTH_TOKEN)

modify_response = groww.modify_smart_order(
    smart_order_id="gtt_91a7f4",
    smart_order_type=groww.SMART_ORDER_TYPE_GTT,
    segment=groww.SEGMENT_CASH,
    quantity=12,
    trigger_price="3988.00",
    trigger_direction=groww.TRIGGER_DIRECTION_DOWN,
    order={
        "order_type": groww.ORDER_TYPE_LIMIT,
        "price": "3985.00",
        "transaction_type": groww.TRANSACTION_TYPE_BUY
    }
)
print(modify_response)
```

Parameters (GTT)

Name	Type	Description
<code>smart_order_id</code> *	str	Smart order identifier. Example: <code>gtt_91a7f4</code> .
<code>smart_order_type</code> *	str	Set to <code>GTT</code> (required for routing).
<code>segment</code> *	str	Segment of the order (required for routing). See Segment . Examples: <code>CASH</code> , <code>FNO</code> .
<code>quantity</code>	int	Updated quantity.
<code>trigger_price</code>	str	Updated trigger price (decimal string).
<code>trigger_direction</code>	str	Updated trigger direction. Examples: <code>UP</code> , <code>DOWN</code> .
<code>order["order_type"]</code>	str	Updated order type. See Order type .
<code>order["price"]</code>	str	Updated limit price (required if <code>order_type</code> is <code>LIMIT</code> or <code>SL</code> ; set to <code>None</code> for <code>MARKET</code> / <code>SL_M</code>).
<code>order["transaction_type"]</code>	str	Transaction type (required but not modifiable). See Transaction type .
<code>child_legs</code>	dict	Updated child legs for bracket orders (all child leg fields modifiable if provided).

* required parameters

Modify OCO

```
from growwapl import GrowwAPI

# Groww API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

# Initialize Groww API
groww = GrowwAPI(API_AUTH_TOKEN)

modify_oco_response = groww.modify_smart_order(
    smart_order_id="oco_a12bc3",
    smart_order_type=groww.SMART_ORDER_TYPE_OCO,
    segment=groww.SEGMENT_FNO,
    quantity=40,
    duration=groww.VALIDITY_DAY,
    product_type=groww.PRODUCT_MIS,
    target={
        "trigger_price": "122.00"
    },
    stop_loss={
        "trigger_price": "97.50"
    }
)
print(modify_oco_response)
```

Parameters (OCO)

Name	Type	Description
<code>smart_order_id</code> *	str	Smart order identifier. Example: <code>oco_a12bc3</code> .
<code>smart_order_type</code> *	str	Set to <code>OCO</code> (required for routing).
<code>segment</code> *	str	Segment of the order (required for routing). See Segment . Examples: <code>FNO</code> , <code>CASH</code> .
<code>duration</code>	str	Updated validity. See Validity .
<code>quantity</code>	int	Updated total quantity.
<code>product_type</code>	str	Updated product. See Product type .
<code>target["trigger_price"]</code>	str	Updated target trigger price (decimal string).
<code>target["order_type"]</code>	str	Target order type (not modifiable). See Order type .
<code>target["price"]</code>	str	Target limit price (not modifiable).
<code>stop_loss["trigger_price"]</code>	str	Updated stop-loss trigger price (decimal string).
<code>stop_loss["order_type"]</code>	str	Stop-loss order type (not modifiable). See Order type .
<code>stop_loss["price"]</code>	str	Stop-loss limit price (not modifiable).

* required parameters

Response

```
{
  "smart_order_id": "oco_a12bc3",
  "smart_order_type": "OCO",
```

```
    "status": "ACTIVE",
    "quantity": 40
}
```

Schema reference: [Smart Order Response \(GTT\)](#) or [Smart Order Response \(OCO\)](#).

Modify vs Cancel-Create

What you need to change	GTT	OCO	Action
Quantity	✔ Modify	✔ Modify	Use modify
Trigger price	✔ Modify	✔ Modify (both legs)	Use modify
Trigger direction	✔ Modify	N/A	Use modify
Order type	✔ Modify	✘ Not modifiable	GTT: modify; OCO: cancel+create
Limit price	✔ Modify	✘ Not modifiable	GTT: modify; OCO: cancel+create
Duration/validity	✘ Not modifiable	✔ Modify	OCO: modify; GTT: cancel+create
Product type	✘ Not modifiable	✔ Modify	OCO: modify; GTT: cancel+create
Symbol/contract	✘ Not modifiable	✘ Not modifiable	Cancel + create
Exchange	✘ Not modifiable	✘ Not modifiable	Cancel + create
Segment	✘ Not modifiable	✘ Not modifiable	Cancel + create
Smart order type (GTT ↔ OCO)	✘ Not modifiable	✘ Not modifiable	Cancel + create

"Note: When a field is marked as "Not modifiable" (✘), you must cancel the existing smart order and create a new one with the desired changes. The "N/A" designation indicates that the feature does not apply to that smart order type."

Cancel Smart Order

Cancel any active smart order.

Python SDK Usage

```
from growwap1 import GrowwAPI

# Groww API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

# Initialize Groww API
groww = GrowwAPI(API_AUTH_TOKEN)

cancel_response = groww.cancel_smart_order(
    segment=groww.SEGMENT_CASH,
    smart_order_type=groww.SMART_ORDER_TYPE_GTT,
    smart_order_id="gtt_91a7f4"
)
print(cancel_response)
```

Parameters

Name	Type	Description
segment *	str	Segment of the smart order to cancel. See Segment . Examples: CASH , FMO .
smart_order_type *	str	Smart order type. Examples: GTT , OCO .
smart_order_id *	str	Smart order identifier. Example: gtt_91a7f4 .

* required parameters

Response

```
{
  "smart_order_id": "gtt_91a7f4",
  "smart_order_type": "GTT",
  "status": "CANCELLED"
}
```

Schema reference: [Smart Order Response \(GTT\)](#) or [Smart Order Response \(OCO\)](#).

Get Smart Order

Retrieve details of a specific smart order by its internal ID.

Python SDK Usage

```
from growwap1 import GrowwAPI

# Groww API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

# Initialize Groww API
groww = GrowwAPI(API_AUTH_TOKEN)

order_details = groww.get_smart_order(
    segment=groww.SEGMENT_CASH,
    smart_order_type=groww.SMART_ORDER_TYPE_GTT,
    smart_order_id="gtt_91a7f4"
)
print(order_details)
```

Parameters

Name	Type	Description
segment *	str	Segment of the smart order to fetch. See Segment . Examples: CASH , FMO .
smart_order_type *	str	Smart order type. Examples: GTT , OCO .
smart_order_id *	str	Smart order identifier. Example: gtt_91a7f4 .

* required parameters

Response

```
{
```

```

    "smart_order_id": "gtt_91a7f4",
    "smart_order_type": "GTT",
    "status": "ACTIVE",
    "trading_symbol": "TCS",
    "exchange": "NSE",
    "quantity": 10,
    "product_type": "CMC",
    "duration": "DAY",
    "order": {
      "order_type": "LIMIT",
      "price": "3990.00",
      "transaction_type": "BUY"
    },
    "trigger_direction": "DOWN",
    "trigger_price": "3985.00",
    "segment": "CASH",
    "ltp": 4000.50,
    "remark": null,
    "display_name": "TCS Ltd",
    "child_legs": null,
    "is_cancellation_allowed": true,
    "is_modification_allowed": true,
    "created_at": "2025-09-30T07:00:00",
    "expiry_at": "2025-09-30T07:00:00",
    "triggered_at": null,
    "updated_at": "2025-09-30T07:00:00"
  }
}
```

Schema reference: [Smart Order Response \(GTT\)](#) or [Smart Order Response \(OCO\)](#).

List Smart Orders

Filter and list smart orders by type, segment, status and a time window. Pagination is supported.

Python SDK Usage

```

from growapi import GrowAPI

# Grow API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

# Initialize Grow API
grow = GrowAPI(API_AUTH_TOKEN)

orders_list = grow.get_smart_order_list(
    segment=grow.SEGMENT_FNO,
    smart_order_type=grow.SMART_ORDER_TYPE_OCO,
    status=grow.SMART_ORDER_STATUS_ACTIVE,
    page=0,
    page_size=10,
    start_date_time="2025-01-16T09:15:00",
    end_date_time="2025-01-16T15:30:00"
)

print(orders_list)
```

Parameters

Name	Type	Description
segment	str	Segment to list smart orders for. See Segment . Examples: FNO , CASH . Optional; server defaults may apply if omitted.
smart_order_type	str	Smart order type to list. Examples: OCO , GTT . Optional; server defaults may apply if omitted.
status	str	Current state filter. Examples: ACTIVE , TRIGGERED , CANCELLED , COMPLETED . Optional; server defaults may apply if omitted.
page	int	Page number starting from 0. Use with <code>page_size</code> to paginate long lists. Min: 0 , Max: 500 .
page_size	int	Number of records per page. Tune this for your UI/export. Min: 1 , Max: 50 .
start_date_time	str	Inclusive start of the time window in ISO 8601. Examples: 2025-01-16T09:15:00 , 2025-01-16T00:00:00 .
end_date_time	str	Inclusive end of the time window in ISO 8601. Must not be before <code>start_date_time</code> . Examples: 2025-01-16T09:15:00 , 2025-01-16T00:00:00 .

Validations:

- `end_date_time` must not be before `start_date_time` .
- Date range between `start_date_time` and `end_date_time` must not exceed one month.

“Tip If you expect an order that has already been triggered or cancelled, switch the `status` filter accordingly—`ACTIVE` only returns live, untriggered smart orders.”

Response

```

{
  "orders": [
    {
      "smart_order_id": "gtt_91a7f4",
      "smart_order_type": "GTT",
      "status": "ACTIVE",
      "trading_symbol": "TCS",
      "exchange": "NSE",
      "quantity": 10
    }
  ]
}
```

Name	Type	Description
orders	list	List of smart orders
orders[]	dict	Smart order item (GTT or OCO). See Smart Order Response (GTT) or Smart Order Response (OCO)

Smart Order Constants

The Python SDK provides constants for smart order types, trigger directions, and statuses:

Smart Order Types

```

grow.SMART_ORDER_TYPE_GTT # "GTT"
grow.SMART_ORDER_TYPE_OCO # "OCO"
```

Trigger Directions

```

grow.TRIGGER_DIRECTION_UP # "UP"
grow.TRIGGER_DIRECTION_DOWN # "DOWN"
```

Smart Order Status

groww.SMART_ORDER_STATUS_ACTIVE	# "ACTIVE" - Order is monitoring trigger conditions
groww.SMART_ORDER_STATUS_TRIGGERED	# "TRIGGERED" - Trigger condition met, order placed
groww.SMART_ORDER_STATUS_CANCELLED	# "CANCELLED" - User cancelled the order
groww.SMART_ORDER_STATUS_EXPIRED	# "EXPIRED" - Order expired due to time/date expiry
groww.SMART_ORDER_STATUS_FAILED	# "FAILED" - Order placement or trigger failed
groww.SMART_ORDER_STATUS_COMPLETED	# "COMPLETED" - Order successfully completed

Quick Tips

- Use a unique `reference_id` per new smart order to avoid accidental duplicates.
- For OCO, ensure `quantity <= abs(net_position_quantity)`. The leg directions are derived from your net position.
- All prices should be passed as decimal strings (e.g., "3985.00").
- Use the SDK constants (like `SMART_ORDER_TYPE_GTT`) for better type safety and code clarity.

Schemas

Smart Order Response (GTT)

Name	Type	Description
smart_order_id	str	Smart order identifier
smart_order_type	str	Smart order type. Example: GTT.
status	str	Smart order status (e.g., ACTIVE).
trading_symbol	str	Trading Symbol of the instrument as defined by the exchange
exchange	str	Stock exchange
quantity	int	Quantity
product_type	str	Product type
duration	str	Validity
order.order_type	str	Order type. See Order type . Examples: LIMIT, SL.
order.price	str	Price for LIMIT/SL
order.transaction_type	str	See Transaction type . Examples: BUY, SELL.
ltp	float	Last traded price of the instrument
trigger_direction	str	Trigger direction. Examples: UP, DOWN.
trigger_price	str	Trigger price (decimal string)
segment	str	Market segment
remark	str	Remark or status message
display_name	str	Display name for the instrument
child_legs	dict	Child legs for bracket orders (target/stop-loss)
is_cancellation_allowed	bool	Whether cancellation is allowed
is_modification_allowed	bool	Whether modification is allowed
created_at	str	Creation time (ISO 8601). Examples: 2025-01-16T09:15:00, 2025-01-16T00:00:00.
expire_at	str	Expiry time (ISO 8601). Examples: 2025-01-16T09:15:00, 2025-01-16T00:00:00.
triggered_at	str	Trigger time (ISO 8601). Examples: 2025-01-16T09:15:00, 2025-01-16T00:00:00.
updated_at	str	Last updated time (ISO 8601). Examples: 2025-01-16T09:15:00, 2025-01-16T00:00:00.

Smart Order Response (OCO)

Name	Type	Description
smart_order_id	str	Smart order identifier
smart_order_type	str	Smart order type. Example: OCO.
status	str	Smart order status (e.g., ACTIVE).
trading_symbol	str	Trading Symbol of the instrument as defined by the exchange
exchange	str	Stock exchange
quantity	int	Quantity of the order to be placed
product_type	str	Product type
duration	str	Validity
target_trigger_price	str	Target trigger price (decimal string)
target_order_type	str	Order type. See Order type . Examples: LIMIT, MARKET.
target_price	str	Target limit price
stop_loss_trigger_price	str	Stop-loss trigger price (decimal string)
stop_loss_order_type	str	Order type. See Order type . Examples: SL, SL_M.
stop_loss_price	str	Stop-loss limit price
is_cancellation_allowed	bool	Whether cancellation is allowed
is_modification_allowed	bool	Whether modification is allowed
created_at	str	Creation time (ISO 8601). Examples: 2025-01-16T09:15:00, 2025-01-16T00:00:00.
expire_at	str	Expiry time (ISO 8601). Examples: 2025-01-16T09:15:00, 2025-01-16T00:00:00.
triggered_at	str	Trigger time (ISO 8601). Examples: 2025-01-16T09:15:00, 2025-01-16T00:00:00.
updated_at	str	Last updated time (ISO 8601). Examples: 2025-01-16T09:15:00, 2025-01-16T00:00:00.